Tarea 5 - Estructuras de Datos, Divide y Vencerás

Análisis y Diseño de Algoritmos — Maestría PCIC (Semestre 2027-1)

Tlacaelel Jaime Flores Villaseñor Victor Rosales Jaimes

2026-11-08

1 Ejercicio 1. Heap binario máximo y heapsort

1.1 Enunciado

Considera los algoritmos de heap binario máximo y heapsort que aparecen en las diapositivas utilizadas en clase.

1.1.1 Inciso a. Propiedad de `sink`

Demuestra la siguiente propiedad de la función `sink` (diapositiva 20): si al llamar `sink(A, k, n)`, cada uno de los subárboles con raíces $A[2k]$ y $A[2k+1]$, respectivamente, cumple con la propiedad heap-max (es decir, el entero en cada vértice del árbol es mayor o igual que los enteros en sus hijos), entonces al finalizar `sink`, el subárbol con raíz $A[k]$ cumple con la propiedad heap-max.

1.2 Solución del inciso a

1.2.1 Funcionamiento de `sink`

La función `sink(A, k, n)` restaura la propiedad heap-max en el subárbol cuya raíz está en la posición k, suponiendo que los subárboles de sus hijos ya cumplen dicha propiedad. Mientras el nodo actual tenga al menos un hijo, se selecciona al hijo de mayor valor y se compara con el padre. Si el padre es mayor o igual que ese hijo, el procedimiento termina, pues también es mayor o igual que cualquier otro hijo existente. En caso contrario, se intercambian ambos valores y se continúa desde la posición del hijo seleccionado, a la que descendió el valor del padre. Este proceso se repite hasta llegar a una hoja o hasta que el nodo actual cumpla la propiedad heap-max con sus hijos. Al finalizar, todo el subárbol de la posición inicial satisface dicha propiedad.

1.2.2 Demostración

La demostracion se hace por inducción.

- Caso base: Si $h = 0$ el subarbol consiste únicamente en una hoja, como esta no tiene hijos cumple con la propiedad de heap-max y `sink` termina sin realizar cambios.
- Hipótesis de induccion: Suponemos que la afirmación es cierta para todo los subárboles con altura menor a h.
- Paso inductivo: Consideremos un subárbol T de altura $h > 0$, cuya raíz está en la posición k y contiene el valor $x = A[k]$. Por la condición del enunciado, los subárboles que comienzan en los hijos de esta raíz ya cumplen la propiedad heap-max.

Sea j la posición del hijo de mayor valor y sea $y = A[j]$. Si la raíz tiene un solo hijo, seleccionamos ese hijo. Entonces tenemos dos casos:

1. Si $x \geq y$, la raíz es mayor o igual que cada uno de sus hijos. Como los subárboles de estos ya son heaps máximos, todo T cumple la propiedad heap-max y `sink` termina.
2. Si $x < y$, el algoritmo intercambia ambos valores: y queda en la raíz de T y x queda en la posición j y el procedimiento continúa desde j.

En el segundo caso, consideremos el subárbol cuya raíz está en j. Su altura es menor que h. El intercambio solo cambió el valor de su raíz; los subárboles de los hijos de j permanecieron intactos y siguen siendo heaps máximos. Por lo tanto, por la hipótesis de inducción, dicho subárbol queda convertido en un heap-max.

Falta comprobar que el valor y, que subió a la raíz de T, queda correctamente colocado. Antes del intercambio, y era el máximo de su propio subárbol. Además, si existe otro hijo de la raíz de T, y es mayor o igual que su valor y, por tanto, que todos los valores de ese otro subárbol. Como también $y > x$, resulta que y es mayor o igual que todos los valores que quedan debajo de él. Los intercambios posteriores únicamente reorganizan esos valores, por lo que esta relación se conserva.

Así, al finalizar, los subárboles de ambos hijos existentes son heap-max y la raíz de T es mayor o igual que sus hijos. Por consiguiente, todo T cumple la propiedad heap-max.

1.3 Enunciado del inciso b

Demuestra la siguiente invariante de ciclo de la fase de construcción de heapsort (ciclo de la diapositiva 42): al inicio del ciclo, para todo $k + 1 \leq \ell \leq n$, se cumple que el subárbol con raíz en $A[\ell]$ cumple con la propiedad heap-max.

1.4 Solución del inciso b

Demostraremos que, al inicio de cada iteración, para toda posición ℓ tal que $k + 1 \leq \ell \leq n$, el subárbol con raíz en $A[\ell]$ cumple la propiedad heap-max.

1.4.1 Inicialización

Antes de la primera iteración, $k = \lfloor n/2 \rfloor$. Todas las posiciones $\ell > \lfloor n/2 \rfloor$ corresponden a hojas, pues la posición de su posible hijo izquierdo, 2ℓ, es mayor que n. Cada hoja cumple la propiedad heap-max porque no tiene hijos. Por tanto, el invariante se cumple al inicio de la primera iteración.

1.4.2 Mantenimiento

Supongamos que el invariante se cumple al inicio de una iteración cuyo índice es k. Los hijos existentes del nodo en la posición k están en las posiciones $2k$ y $2k + 1$, ambas mayores que k. Por el invariante, los subárboles con raíz en esos hijos cumplen la propiedad heap-max.

En consecuencia, se cumplen las condiciones del inciso a: después de ejecutar sink(A,k,n), todo el subárbol con raíz en la posición k es un heap máximo.

Además, los subárboles que comienzan en posiciones mayores que k siguen cumpliendo la propiedad. Los que están dentro del subárbol recién reparado la cumplen porque forman parte de un heap máximo; los que están fuera permanecen sin cambios.

Así, después de ejecutar sink, todos los subárboles con raíz en posiciones ℓ tales que $k \leq \ell \leq n$ cumplen la propiedad heap-max. Al disminuir el índice a $k' = k - 1$, este intervalo equivale a $k' + 1 \leq \ell \leq n$. Por tanto, el invariante se conserva para la siguiente iteración.

1.4.3 Conclusión al terminar

El ciclo termina después de procesar $k = 1$ y disminuir el índice a $k = 0$. Por el invariante, todos los subárboles con raíz en posiciones $1 \leq \ell \leq n$ cumplen la propiedad heap-max. En particular, la cumple el subárbol con raíz en la posición 1, que contiene todo el árbol. Por lo tanto, al finalizar la fase de construcción, el arreglo representa un heap máximo.

2 Ejercicio 2. Emparejamiento de estaciones en un árbol

2.1 Enunciado

Sea T un árbol binario enraizado. Todos los vértices de T tienen un atributo binario `flag`, y solo para un número par de ellos, a los que llamaremos **estaciones**, su atributo es `true`.

Se busca emparejar las estaciones de forma tal que en T existan caminos disjuntos en aristas que conecten las parejas de estaciones. Los caminos no pueden compartir aristas, aunque sí pueden compartir vértices.

Diseña un algoritmo tipo divide y vencerás de tiempo $O(n)$ que imprima parejas de estaciones con la propiedad descrita, donde n es el número de vértices de T. Toda estación debe quedar emparejada.

Puedes suponer que existe una instrucción `Print(s)` que imprime la cadena `s` y toma tiempo $O(1)$. La raíz r de T puede ser parte de la entrada.

El árbol está representado como una lista de adyacencia. Para cada vértice v, left(v) y right(v) denotan sus hijos izquierdo y derecho, y $\pi(v)$ denota su padre. Si alguno no existe, su valor es `NONE`.

Explica la idea del algoritmo y da pruebas de terminación, corrección, tiempo y espacio. ¿Existen entradas para las que tu algoritmo corre en tiempo $O(\log n)$? Argumenta tu respuesta.

2.2 Idea del algoritmo

La idea del algoritmo es aplicar divide y vencerás sobre el árbol con raíz r. Para cada nodo v, se resuelven recursivamente los subárboles de sus hijos. Cada llamada devuelve, como máximo, una estación pendiente de emparejar, o `NONE` si todas las estaciones de su subárbol ya quedaron emparejadas. El nodo v reúne las estaciones pendientes recibidas y se incluye a sí mismo si es una estación. Con estas candidatas forma e imprime las parejas posibles y devuelve al padre la estación que quede pendiente, si existe. Durante este proceso se garantiza que los caminos de las parejas formadas no comparten aristas y que el camino desde la estación pendiente hasta v queda libre de aristas utilizadas por esas parejas. Como el árbol completo contiene un número par de estaciones, al finalizar la llamada sobre r no queda ninguna pendiente y todas quedan emparejadas.

2.3 Pseudocódigo

```
Función Emparejar(v)
    si v = NONE entonces
        devolver NONE

    izquierda ← Emparejar(left(v))
    derecha   ← Emparejar(right(v))

    P ← lista vacía

    si izquierda  NONE entonces
        AgregarAlFinal(P, izquierda)

    si derecha  NONE entonces
        AgregarAlFinal(P, derecha)

    si flag(v) = true entonces
        AgregarAlFinal(P, v)

    si longitud(P)  2 entonces
        Print((P[1], P[2]))
```

```
    si longitud(P) = 1 entonces
        devolver P[1]

    si longitud(P) = 3 entonces
        devolver P[3]

    devolver NONE
```

2.4 Prueba de terminación

Consideremos como medida el número de vértices del subárbol recibido por cada llamada a `Emparejar(v)`. Si $v = \texttt{NONE}$, el subárbol está vacío y la función termina inmediatamente.

Si $v \neq \texttt{NONE}$, el algoritmo realiza llamadas recursivas sobre los subárboles de sus hijos. Cada uno contiene estrictamente menos vértices que el subárbol con raíz en v, pues no incluye al propio v. Por tanto, la medida disminuye estrictamente en cada descenso recursivo y, al ser un entero no negativo, necesariamente se alcanza el caso base.

Una vez que regresan ambas llamadas recursivas, se ejecuta una cantidad finita de operaciones sobre una lista de tamaño máximo tres y se devuelve un resultado. En consecuencia, todas las llamadas terminan y, en particular, termina la llamada inicial `Emparejar(r)`.

2.5 Prueba de corrección

2.5.1 Propiedad que se demostrará

2.5.2 Caso base

2.5.3 Paso inductivo

2.5.4 Conclusión para el árbol completo

2.6 Tiempo de ejecución

2.7 Espacio utilizado

2.8 ¿Existen entradas con tiempo $O(\log n)$?

3 Ejercicio 3. Reconstrucción de un árbol binario

3.1 Enunciado

Considera árboles binarios enraizados, representados como en el ejercicio anterior. En un recorrido se realiza una exploración tipo DFS desde la raíz y se produce una secuencia en la que cada vértice aparece una y solo una vez. Las siguientes políticas se aplican recursivamente:

- **En orden:** subárbol izquierdo, raíz y subárbol derecho.
- **Preorden:** raíz, subárbol izquierdo y subárbol derecho.
- **Postorden:** subárbol izquierdo, subárbol derecho y raíz.

Se reciben un entero n y dos arreglos de longitud n, EO y PO, que contienen los recorridos en orden y en preorden, respectivamente, de un árbol binario enraizado T. El árbol no tiene por qué ser completo ni balanceado.

Diseña un algoritmo tipo divide y vencerás de tiempo $O(n^2)$ que produzca T a partir de n, EO y PO. Explica su idea y da pruebas de terminación, corrección, tiempo y espacio.

¿Existen entradas para las que tu algoritmo corre en tiempo $O(n \log n)$? ¿Y $O(n)$? ¿Cómo modificarías tu algoritmo si PO tuviera el recorrido en postorden en lugar de preorden? Argumenta tus respuestas.

3.2 Idea del algoritmo

3.3 Pseudocódigo

3.4 Prueba de terminación

3.5 Prueba de corrección

3.5.1 Caso base

3.5.2 Hipótesis inductiva

3.5.3 Paso inductivo

3.6 Tiempo de ejecución

3.7 Espacio utilizado

3.8 Entradas con tiempo $O(n \log n)$

3.9 Entradas con tiempo $O(n)$

3.10 Modificación para usar postorden

3.10.1 Cambios en la idea del algoritmo

3.10.2 Pseudocódigo o cambios al pseudocódigo original

3.10.3 Justificación

4 Ejercicio 4. Consultas de rutas a gran escala

4.1 Enunciado

Ve el video indicado en la tarea y explica con tus propias palabras, en **no más de una página**, por qué el algoritmo de Dijkstra no es lo suficientemente rápido en sistemas del tamaño de empresas como Google y cómo puede modificarse para responder con rapidez a pesar de los grandes volúmenes de peticiones que reciben.

4.2 Explicación